



JASHORE UNIVERSITY OF SCIENCE AND TECHNOLOGY

Department of Computer Science and Engineering

LAB REPORT

Course Code: CSE 3102

Course Title: Operating System and System Programming Lab

Lab No: 03

Lab Title: Implementation of Non-Preemptive Shortest Job First (SJF) Scheduling Algorithm

Submitted To:

Monishanker Halder,
Assistant Professor
Dept. of Computer Science and Engineering, JUST

Submitted By:

Name: Adnan Zaman Niloy
Student ID: 210142
Session: 2021-2022
Year/Semester: 3rd Year, 1st Semester

Date of Submission: 2nd July 2025

Experiment Name: Implementation of Non-Preemptive Shortest Job First (SJF) Scheduling Algorithm

1. Objective

To implement and analyze the Non-Preemptive Shortest Job First (SJF) CPU scheduling algorithm and calculate the average waiting time for a given set of processes.

2. Introduction and Theoretical Background

2.1 Problem Statement

Consider the following set of processes with their arrival time and CPU burst time (in milliseconds):

Process	Arrival Time	Burst Time
P1	0	12
P2	2	3
P3	4	5
P4	5	8
P5	6	2

Using the Shortest Job First (Non-Preemptive) scheduling algorithm:

- Draw the Gantt chart that illustrates the execution order.
- Calculate the waiting time for each process.
- Determine the average waiting time.

2.2 Concept of SJF (Non-Preemptive) Scheduling

The Shortest Job First (SJF) scheduling algorithm selects the process with the shortest CPU burst time among the available (arrived) processes. In the non-preemptive version:

- Once a process starts execution, it runs to completion.
- It provides optimal average waiting time.
- However, it may lead to starvation of longer processes if shorter jobs continue to arrive.

2.3 Algorithm Description

The steps of the **Non-Preemptive SJF** algorithm are:

1. At each time unit, select the process with the **shortest burst time** from the **arrived and incomplete** processes.

2. Execute the selected process **until completion**.
3. Update the **current system time** after the process finishes.
4. Mark the process as **completed**.
5. If no process has arrived, **advance the time** until one becomes available.
6. Repeat until **all processes are executed**.

3. Methodology and Implementation

3.1 Data Structure Architecture

```
struct Process {  
    string name;  
    int arrival;  
    int burst;  
    int start;  
    int finish;  
    int waiting;  
    bool done = false;  
};
```

3.2 Final C++ Code

```
#include <bits/stdc++.h>  
  
using namespace std;  
  
struct Process  
{  
    string name;  
    int arrival, burst, start, finish, waiting;  
    bool done = false;  
};  
  
int main()  
{
```

```

int n = 5;

vector<Process> p(n);

for (int i = 0; i < n; ++i)
    cin >> p[i].name >> p[i].arrival >> p[i].burst;


int completed = 0, currentTime = 0;

float totalWaiting = 0;


while (completed < n)
{
    int idx = -1, minBurst = INT_MAX;

    for (int i = 0; i < n; ++i)
    {
        if (!p[i].done && p[i].arrival <= currentTime && p[i].burst < minBurst)
        {
            minBurst = p[i].burst;

            idx = i;
        }
    }

    if (idx == -1)
    {
        currentTime++;
    }
    else
    {
        p[idx].start = currentTime;

        p[idx].finish = p[idx].start + p[idx].burst;

        p[idx].waiting = p[idx].start - p[idx].arrival;

        totalWaiting += p[idx].waiting;

        currentTime = p[idx].finish;
    }
}

```

```

        p[idx].done = true;

        completed++;
    }
}

cout << "\nProcess\tArrival\tBurst\tWaiting\n";
for (auto &proc : p)
{
    cout << proc.name << "\t" << proc.arrival << "\t" << proc.burst << "\t" << proc.waiting <<
    "\n";
}

cout << fixed << setprecision(2);
cout << "Average Waiting Time: " << totalWaiting / n << "\n";

return 0;
}

```

4. Test Case Analysis

4.1 Input Data

Process	Arrival Time	Burst Time
P1	0	12
P2	1	3
P3	2	5
P4	3	8
P5	4	2

4.2 Gantt Chart Execution Timeline

1. **Time 0–12:** P1 executes (only process available initially).
2. **Time 12–14:** P5 executes (shortest among P2, P3, P4, P5).
3. **Time 14–17:** P2 executes (shortest among remaining).
4. **Time 17–22:** P3 executes.
5. **Time 22–30:** P4 executes.

Gantt Chart:



4.3 Output Table

Process	Arrival Time	Burst Time	Start Time	Finish Time	Waiting Time
P1	0	12	0	12	0
P2	1	3	14	17	13
P3	2	5	17	22	15
P4	3	8	22	30	19
P5	4	2	12	14	8

4.4 Average Waiting Time Calculation

Average Waiting Time = $0+13+15+19+8 \div 5 = 55/5 = 11.00$

5. Results and Discussion

- P1 executes first with zero waiting time, as it arrives earliest.
- Shorter processes (P2, P3, P5) are delayed due to P1's long execution.
- The SJF algorithm significantly reduces average waiting time (11.00 units in this case).
- It may suffer from the convoy effect, where shorter jobs wait behind a long job.
- Starvation is not an issue in this static dataset, but in dynamic systems with frequent short jobs, longer ones may starve.

6. Conclusion

The Non-Preemptive Shortest Job First (SJF) algorithm effectively demonstrates how CPU scheduling can be optimized to minimize average waiting time. It is ideal for batch systems where turnaround efficiency is prioritized.

However, it lacks the flexibility to preempt running processes, which may delay shorter jobs that arrive later. For such dynamic or real-time systems, preemptive algorithms like Shortest Remaining Time First (SRTF) are more suitable.